University of Exeter

ECM2433: The C Family
C++ - Error handling and lambda

C vs. C++: Error handling

University of Exeter

Moving from fragmented manual checks to unified, automated exception management.

C: Fragmented Manual Checks

```
#include <errno.h>
FILE *f = fopen("data.txt", "r");
if (f == NULL) {
    fprintf(stderr, "Error %d:
%s\n", errno, strerror(errno));
    return ERR_OPEN_FAIL;
}
if (process(f) == -1) {
    fclose(f);
    return ERR_PROC_FAIL;
}
fclose(f);
return 0;
```

Logic buried in if(error) blocks

C++: Unified Exception Flow

```
#include <exception>
try {
    FileHandler f("data.txt");
    f.process();
    f.upload();
} catch (const std::exception& e) {
    // error handling here
    log_error(e.what());
}
```

Separates "Good Path" from "Error Path".

Try, catch, throw mechanism

Try

A scope where errors might occur.

Catch

Handle the specific error type.

Throw

Signal a problem that cannot be handled locally.

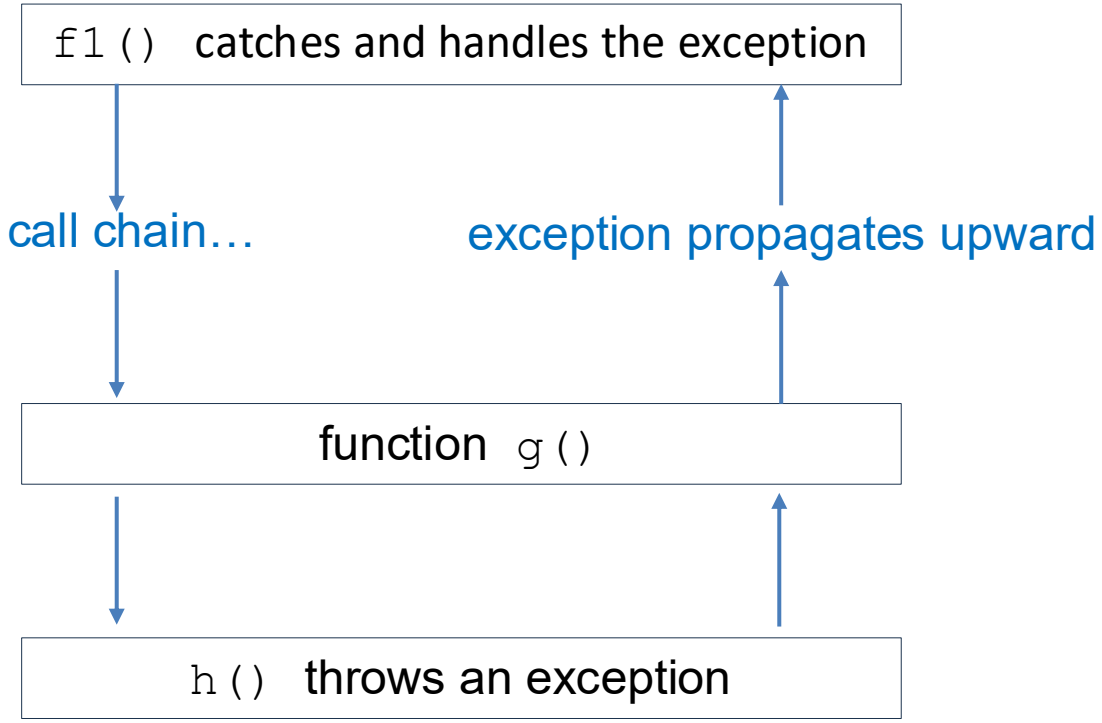
```
void taskmaster(){
    try {
        auto result = do_task();
    } catch (const someError& e) {
        // handle failure
    }
}
int do_task(){
    if (error_condition)
        throw someError();
    return result;
}
```

Try, catch, throw mechanism

```
#include<iostream>
using namespace std;


int divide(int a, int b)
{
    if (b == 0) {
       throw a;
    }
    return a / b;
}


int main()
{
    try {
       cout << "5 / 2 = " << divide(5, 2) << endl;
       cout << "5 / 0 = " << divide(5, 0) << endl;
       cout << "7 / 1 = " << divide(7, 1) << endl;
    } catch (int e) {
       cerr << e << " is divided by zero!" << endl;
    }


    cout << "Program continues..." << endl;
    return 0;
}
```

```
5 / 2 = 2
5 / 0 = 5 is divided by zero!
Program continues...
```

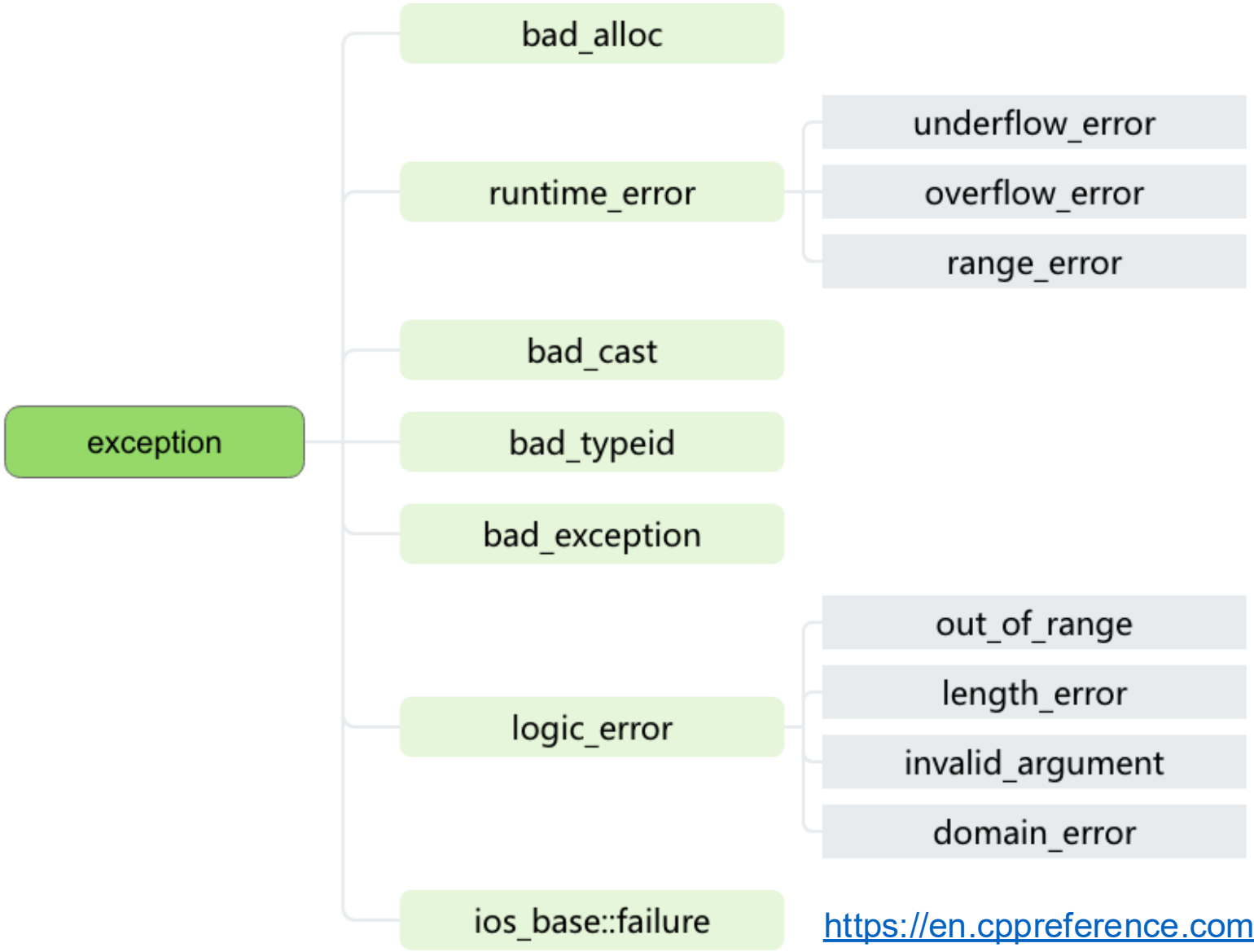
Exception handling




```
void f1(){
    try {
        auto result = do_something();
    } catch (const someError& e) {
        // handle failure
    }
}
```

When an exception is thrown, it propagates up the call stack until a matching catch handler is found.

Kinds of exceptions



Unforeseeable: Environmental/ Input issues

Preventable by code logic.

https://en.cppreference.com/w/cpp/error/exception.html

Kinds of exceptions in std

Exception class	Header file	Meaning
bad_alloc	new	Thrown when dynamic memory allocation using new fails
bad_cast	typeinfo	Thrown when a dynamic_cast fails
bad_typeid	typeinfo	Thrown when typeid(*p) is applied to a null pointer
bad_exception	exception	Thrown when an unexpected exception occurs that violates the function's exception specification, and bad_exception is listed in the exception specification
ios_base::failure	ios	Indicates an error during C++ input/output stream operations
underflow_error	stdexcept	Arithmetic underflow error
overflow_error	stdexcept	Arithmetic overflow error
range_error	stdexcept	Error due to result being outside the valid range during internal computation
out_of_range	stdexcept	Thrown when a value is outside the allowed range
length_error	stdexcept	Thrown when attempting to create an object exceeding its maximum allowed size
invalid_argument	stdexcept	Thrown when an invalid argument is passed to a function
domain_error	stdexcept	Thrown when a mathematical domain error occurs (input does not meet required preconditions)

Throwing standard exception

```
#include <iostream>
#include <vector>
#include <stdexcept>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3};
    try {
          // Accessing out of bound element
          v.at(10);
    }
    catch (const out_of_range& e) {
          cout << "Caught: " << e.what()<<endl;
    }

    return 0;
}
```

Every standard exception class provides a `what()` method that returns an error message.

```
Caught: vector::_M_range_check: __n (which is 10) >= this->size() (which is 3)
```

Catch multiple exceptions

```
try{
    // code that might throw
} catch(const std::invalid_argument& e){
    // specific handling
} catch(const std::out_of_range& e){
    // specific handling
} catch(const std::exception& e) {
    // general handling
}
```

multiple errors may occur

Catch by const reference

Avoids copying and accidental modification

Order matters

Catch specific derived classes BEFORE base classes.

If none of the catch statements match, the program calls `std::terminate()` and terminates.

Good practice of exception handling

- Don't use exceptions for regular control flow. Only use them for exceptional situations (errors).
- Catch exceptions by reference whenever possible to avoid copying overhead and enable polymorphic selection.
- Always catch exceptions by reference to a constant to avoid accidental modification.
- Always ensure exceptions are caught and handled properly to maintain stability.

Lambda (anonymous) function

Lambdas functions are functions that capture state from an enclosing scope

Capture Clause
(environment)

Trailing Return Type
(Optional)

```
// Basic Syntax
auto myLambda = [](int a , int b ) -> int {
        return a + b ;
};
```

Function body

Parameters (input)

```
// Usage
int result = myLambda(5, 3);  //8
```

Lambda: capturing scope

How the lambda accesses variables from the surrounding environment.

```
int x = 10;
int y = 7;
auto multiplier = [x] (int n) {
     return n * x;
};
int result = multiplier(5); // 50
```

The lambda holds a copy of ‘x'

```
[x](arguments)        // captures x by value (makes a copy)
[&x](arguments)       // captures x by reference
[x, y](arguments)     // captures x, y by value
[&](arguments)        // captures everything by reference
[&, x](arguments)     // captures everything except x by reference
[=](arguments)        // captures everything by value
```

Lambda: more usage

```
bool isEven(int x)
{
    return x % 2 == 0;
}

int main()
{
    std::vector<int> myNums = {1, 6, 5, 8, 3, 5, 4};
    int numEvens = std::count_if(myNums.begin(), myNums.end(), &isEven);
    std::cout << "num evens = " << numEvens << std::endl;

    return 0;
}
```

```
int numEvens = std::count_if(myNums.begin(), myNums.end(), [](int x){return x%2 == 0;});
```

Summary

- C++ exception handling provides a structured way to manage errors using **try, throw, and catch**, separating normal logic from error handling.
- Exceptions **propagate up the call stack** until a matching catch handler is found, allowing errors to be handled at an appropriate level.
- Standard exception classes (e.g. `out_of_range, invalid_argument`) provide useful predefined error types and error messages via what().
- **Lambda functions** allow us to define anonymous functions inline, making code shorter and more flexible.